Programación Concurrente

Clase 10



Facultad de Informática
UNLP

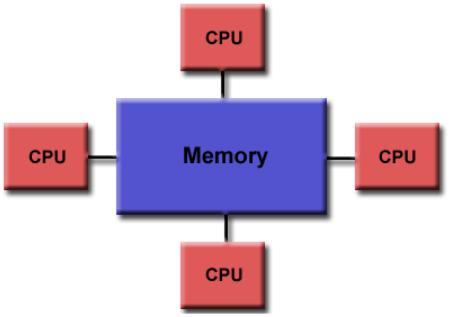
Arquitecturas Paralelas

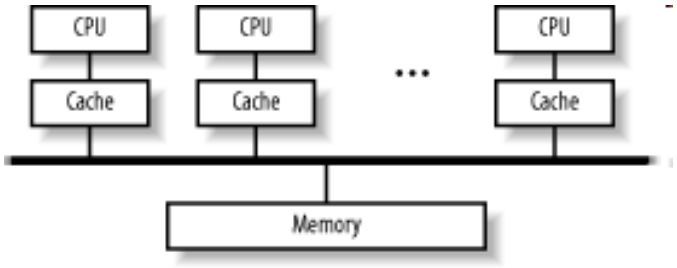
Clasificación por el Espacio de Direcciones

- Las arquitecturas paralelas se clasifican según su espacio de direcciones en:
  - Memoria Compartida.
  - Memoria Distribuida.
- Esta clasificación se relaciona con el modelo de comunicación a utilizar:
  - Accesos a Memoria Compartida (memoria compartida).
  - Intercambio de mensajes (principalmente memoria distribuida).
- En algunos casos también tenemos en la misma plataforma ambos mecanismos → Memoria híbrida

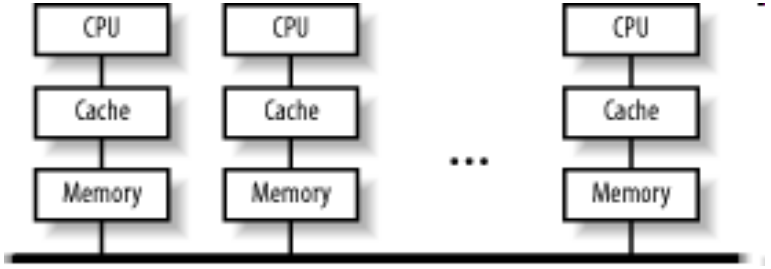
Clasificación por el Espacio de Direcciones

- Multiprocesadores de memoria compartida.
  - Esquemas UMA con bus o crossbar switch (SMP, multiprocesadores simétricos). Problemas de sincronización y consistencia.
  - Esquemas NUMA para mayor número de procesadores distribuidos.
  - Problema de coherencia.





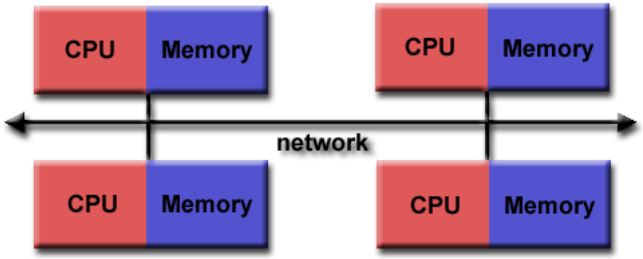
Esquema UMA



Esquema NUMA

Clasificación por el Espacio de Direcciones

- Multiprocesadores con memoria distribuida.
  - Procesadores conectados por una red.
  - Memoria local (no hay problemas de consistencia).
  - Interacción es sólo por pasaje de mensajes.
  - Grado de acoplamiento de los procesadores:
    - Multicomputadores (máquinas fuertemente acopladas). Procesadores y red físicamente cerca. Pocas aplicaciones a la vez, cada una usando un conjunto de procesadores. Alto ancho de banda y velocidad.
    - Clusters.
    - Redes (multiprocesador débilmente acoplado).

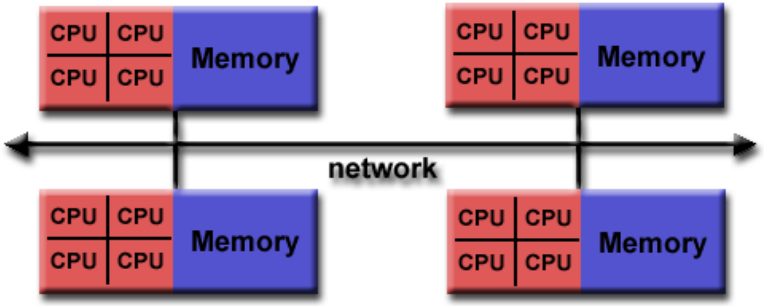






Clasificación por el Espacio de Direcciones

- Debido al surgimiento de los procesadores multicore, los sistemas más grandes y rápidos del mundo de la actualidad combinan características de ambos modelos
- Múltiples máquinas de memoria compartida son inter-conectadas entre sí para permitir que sus procesadores puedan comunicarse





Diseño de Algoritmos Paralelos

Diseño de algoritmos paralelos

- Un algoritmo secuencial es esencialmente una receta o secuencia de pasos para resolver un determinado problema empleando una única unidad de procesamiento.
- En forma similar, un algoritmo paralelo se puede ver como una receta que indica cómo resolver un problema *empleando múltiples unidades de procesamiento*.
- Desarrollar un algoritmo paralelo es más que determinar los pasos para resolver el problema → el programador debe al menos considerar la concurrencia y especificar qué pasos pueden llevarse a cabo simultáneamente (esencial para obtener beneficios)

Diseño de Algoritmos Paralelos

- Para diseñar un algoritmo paralelo se deben realizar alguno de los siguientes pasos:
  - Identificar porciones de trabajo (tareas) concurrentes.
  - Mapear tareas a procesos en distintos procesadores.
  - Distribuir datos de entrada, intermedios y de salida.
  - Manejo de acceso a datos compartidos.
  - Sincronizar procesos.

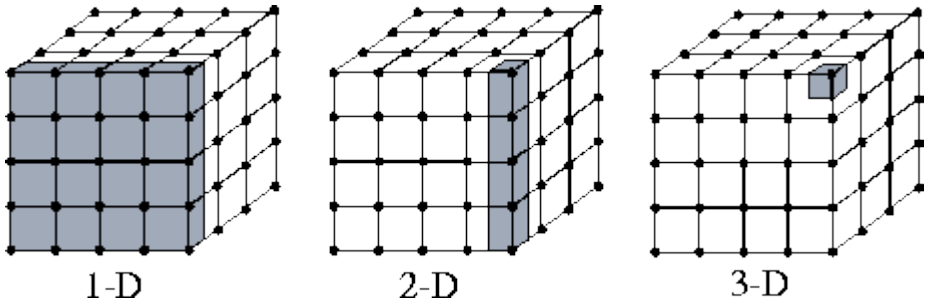
- Pasos Fundamentales: ***Descomposición en Tareas*** y ***Mapeo de Procesos a Procesadores***.

Descomposición en tareas

- Para desarrollar un algoritmo paralelo el primer punto es descomponer el problema en sus componentes funcionales concurrentes (procesos/tareas).
- Se trata de definir un gran número de pequeñas tareas para obtener una descomposición de grano fino, para brindar la mayor flexibilidad a los algoritmos paralelos potenciales → Posteriormente se puede hacer una etapa de aglomeración para incrementar su granularidad.
- Esta descomposición puede realizarse de muchos modos. Un primer concepto es pensar en tareas de igual código (normalmente ***paralelismo de datos o dominio***) pero también podemos tener diferente código (***paralelismo funcional***) o ambas cosas**.**

Descomposición de datos

- Consiste en descomponer los datos asociados a un problema en pequeñas porciones (usualmente del mismo tamaño) y luego asociarle el cómputo relacionado a las mismas para generar las tareas.
- Esta división llevará a un número determinado de tareas, donde cada una comprende de algunos datos y operaciones a realizar sobre los mismos
- Una operación puede requerir datos de diferentes tareas, lo que implicará comunicación y sincronización
- Diferentes particiones son posibles de acuerdo a la estructura de datos disponible

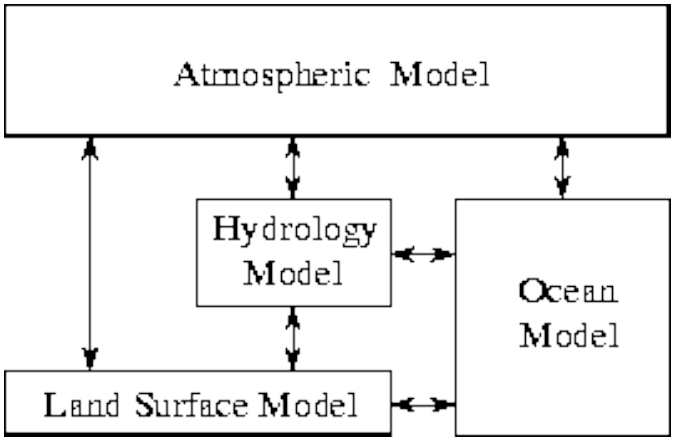


Podría representar el estado de la atmósfera en un modelo climático o un espacio tridimensional en un problema de procesamiento de imágenes

Descomposición funcional

- Se enfoca en el cómputo a realizar más que en los datos → Divide al cómputo en tareas disjuntas y luego examina los datos
- Los requerimientos de datos pueden ser disjuntos (caso ideal) o superponerse significativamente (peor caso → comunicación requerida para evitar replicación de datos).
- La descomposición de datos es la más antigua y, a su vez, la más usada. De todas formas, la descomposición funcional tiene valor como una forma diferente de pensar los problemas
  - Enfocarse en el cómputo a realizar facilita la estructuración del programa y el descubrimiento de oportunidades de optimización (situación no tan obvia cuando uno se enfoca en los datos)

Modelo climático
(1) El modelo atmosférico genera datos sobre la velocidad del viento que son usados por el modelo oceánico; (2) el modelo oceánico genera datos sobre la temperatura de la superificie que son usados por el modelo atomesférico; (3) y así siguiendo…

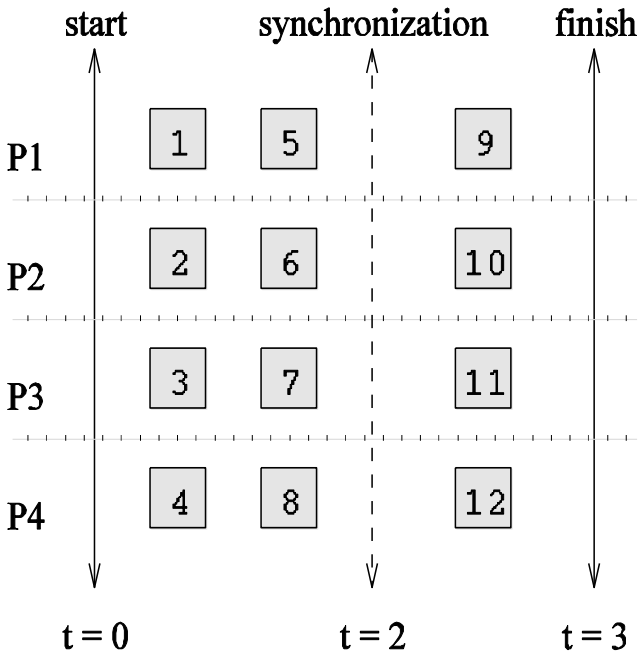


Mapeo de tareas a procesadores

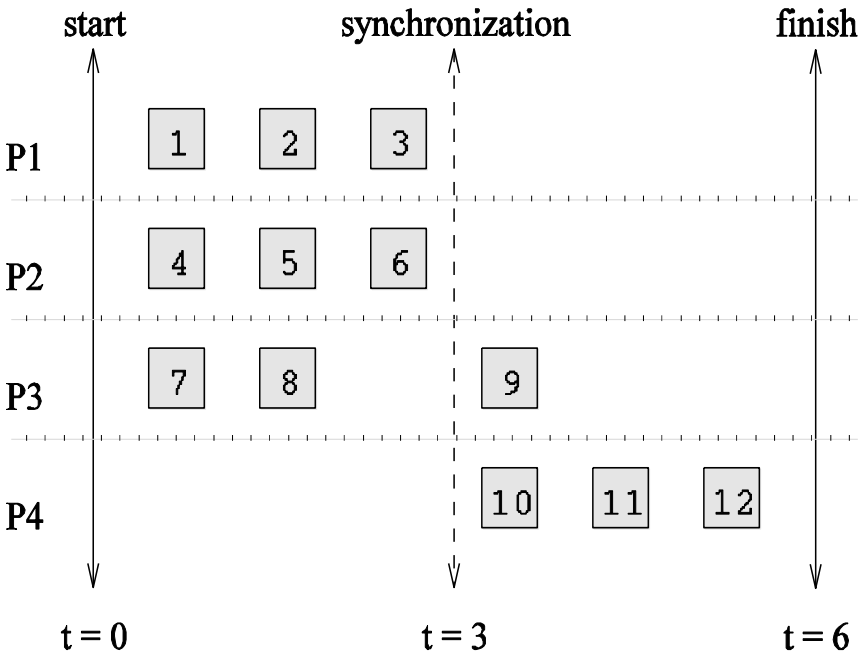
- Se especifica dónde ejecuta cada tarea.
- Este problema no existe en uniprocesadores o máquinas de memoria compartida con scheduling de tareas automático.
- **Objetivo:** minimizar tiempo de ejecución. Dos estrategias, que a veces conflictúan: ubicar tareas que pueden ejecutar concurrentemente en ≠ procesadores para mejorar la concurrencia o poner tareas que se comunican con frecuencia en = procesador para incrementar la localidad.
- El problema es NP-completo: no existe un algoritmo de tiempo polinomial tratable computacionalmente para evaluar *tradeoffs* entre estrategias en el caso general. Existen heurísticas para clases de problema.

Mapeo de tareas a procesadores

Tener en cuenta que un balance de carga equilibrado, NO asegura un mejor mapeo (solución más rápida), además debe lograr que trabajen al mismo tiempo:



(a)



(b)

Métricas de Rendimiento

Métricas del paralelismo

- En el mundo serial la performance con frecuencia es medida teniendo en cuenta los requerimientos de tiempo y memoria de un programa.
- Históricamente, en un algoritmo paralelo para resolver un problema interesa saber cuál es la ganancia en performance.
- Hay otras medidas que deben tenerse en cuenta siempre que favorezcan a sistemas con mejor tiempo de ejecución. Actualmente, interesan otros aspectos como eficiencia energética y esfuerzo de programación.
- A falta de un modelo unificador de cómputo paralelo, el tiempo de ejecución depende del tamaño de la entrada y de la arquitectura y número de procesadores (*sistema paralelo = algoritmo + contexto de software y hardware*).

Métricas del paralelismo

- La diversidad torna complejo el análisis de performance...
  - ¿Qué interesa medir?
  - ¿Qué indica que un sistema paralelo es mejor que otro?
  - ¿Qué sucede si agrego procesadores?
- En la medición de performance es usual elegir un problema y testear el tiempo variando el número de procesadores. Aquí subyacen las nociones de speedup y eficiencia, y la *ley de Amdahl*.
- Otro tema de interés es la *escalabilidad,* que da una medida de usar eficientemente un número creciente de procesadores.

Métricas del paralelismo
Speedup (S)

- S es el cociente entre el tiempo de ejecución del algoritmo serial conocido más rápido (T_s) y el tiempo de ejecución paralelo del algoritmo elegido (T_p):

$$S = \frac{T_S}{T_P}$$

- Refleja el beneficio de emplear procesamiento paralelo para resolver un problema dado comparado a realizarlo en forma secuencial
- Es una medida de cuántas veces más rápido pudimos resolver el problema empleando el algoritmo paralelo con p unidades de procesamiento comparado al algoritmo secuencial.

Métricas del paralelismo
Speedup (S)

- Speedup óptimo depende de la arquitectura (en homogénea P).

$$S_{óptimo} = \sum_{i=0}^{P} \frac{PotenciaCalculo(i)}{PotenciaCalculo(mejor)}$$

- Rango de valores: en general entre *0* y $S_{óptimo}$
- Speedup lineal o perfecto, sublineal y superlineal.

Métricas del paralelismo
Eficiencia (E)

- Cociente entre Speedup y Speedup Óptimo.

$$E = \frac{S}{S_{óptimo}}$$

- Mide la fracción de tiempo en que los procesadores son *útiles* para el cómputo.

- El valor está entre 0 y 1, dependiendo de la efectividad de uso de los procesadores. Cuando es 1 corresponde al speedup perfecto.

Escalabilidad de los Sistemas Paralelos

- Es muy difícil extrapolar la perfomance de un sistema paralelo, a partir de configuraciones con pocos procesadores y conjuntos de datos reducidos.
  - No sirven los estudios con 2, 4, 8 procesadores que proyectan el *Sp* alcanzable con 128 o 256 procesadores o el tiempo de procesamiento cuando tengamos 100 o 1000 veces más datos… ¿Por qué?
  - Básicamente porque los resultados con pequeños conjuntos de datos están afectados por la localidad en el manejo de la memoria, y los resultados con pocos procesadores porque las comunicaciones no computan los costos relacionados con la distancia entre procesadores y la disminución del ancho de banda efectivo.

Métricas del paralelismo
Factores que limitan el Speedup

- Alto porcentaje de código secuencial (*Ley de Amdahl*).
- Alto porcentaje de entrada/salida respecto de la computación.
- Algoritmo no adecuado (necesidad de rediseñar).
- Excesiva contención de memoria (rediseñar código para localidad de datos).
- Tamaño del problema (puede ser chico, o fijo y no crecer con p).
- Desbalance de carga (produciendo esperas ociosas en algunos procesadores).
- Overhead paralelo: ciclos adicionales de CPU para crear procesos, sincronizar, etc.

Paradigmas de Programación Paralela

Paradigmas de Programación Paralela

- ***Paradigma de programación***: clase de algoritmos que resuelve distintos problemas, pero tienen la misma estructura de control.
- Para cada paradigma puede escribirse un esqueleto algorítmico que define la estructura de control común.
- Dentro de la programación paralela pueden encontrarse paradigmas que permiten encuadrar los problemas en alguno de ellos.
- En cada paradigma, los patrones de comunicación son muy similares en todos los casos.

Paradigmas de Programación Paralela

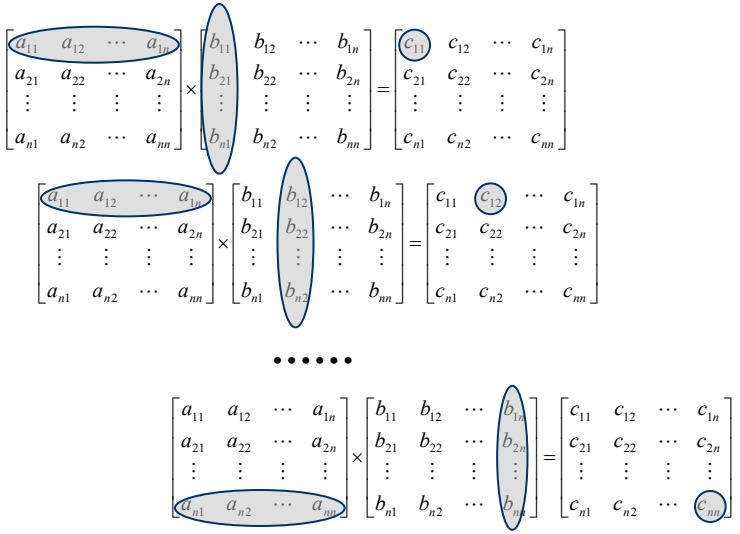
- Cliente / Servidor
- Master/slave o master/worker
- Pipeline y Algoritmos Sistólicos
- Dividir y Conquistar (paralelismo recursive)
- SPMD (paralelismo iterativo)

SPMD

Ejemplo de SPMD: multiplicación de matrices.

Solución secuencial:

```
double a[n,n], b[n,n], c[n,n];
for [i = 1 to n]
  { for [j = 1 to n]
     { c[i,j] = 0;
       for [k = 1 to n]
          c[i,j] = c[i,j] + (a[i,k]*b[k,j]);
     }
  }
```

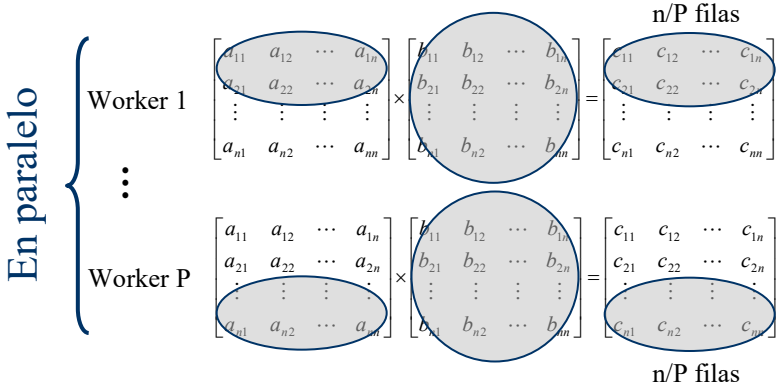


- El loop interno calcula el producto interno de la fila *i* de la matriz *a* por la columna *j* de la matriz *b* y obtiene *c*[*i*,*j*].
- El cómputo de cada producto interno es independiente. Aplicación *embarrasingly parallel* (muchas operaciones en paralelas).
- Diferentes acciones paralelas posibles.

SPMD

Solución paralela por strips:
(*P* procesadores con *P*<*n*)

```
process worker [ w = 1 to P]
  { int primera = (w-1)*(n/P) + 1;
    int ultima = primera + (n/P) - 1;
    for [i = primera to ultima]
      { for [j = 1 to n]
          { c[i,j] = 0;
            for [k = 1 to n]
              c[i,j] = c[i,j] + (a[i,k]*b[k,j]);
          }
      }
  }
```



Ejercicio: a) Si P=8 y n=120. ¿Cuántas asignaciones, sumas y productos hace cada procesador?.
b) Si P1=...=P7 y los tiempos de asignación son 1, de suma 2 y de producto 3; y si P8 es 2 veces más lento. ¿Cuánto tarda el proceso total?. ¿Cuál es el speedup?. ¿Qué puede hacerse para mejorar el speedup?.